

# **Unsafe a Embedded**

“PV281 - Programování v Rustu”

“Marek Vrbka”

# Obsah

|                         |    |
|-------------------------|----|
| Unsafe a Embedded ..... | 1  |
| Unsafe .....            | 3  |
| Úvod do Embedded .....  | 8  |
| Embedded v Rustu .....  | 23 |
| Ekosystém .....         | 29 |

**Unsafe**

# Co je unsafe

- unsafe je klíčové slovo, které označuje blok nebo celý modul, kde kompilátor neprovádí všechny bezpečnostní kontroly.
- Neznamená to, že kód automaticky přestane být bezpečný – stále musíte dodržovat pravidla Rustu ručně.
- Používá se pro operace, které kompilátor nemůže ověřit jako bezpečné – např. dereference surových ukazatelů, volání FFI funkcí, nebo přístup k hardwaru.
- Cílem je izolovat nebezpečný kód a minimalizovat riziko.

# Jak se používá unsafe?

- Označení funkce

```
1 /// invariant: val must be valid pointer to u64
2 unsafe fn foo(val: *mut 64) -> u64 {
3     *val
4 }
```

Rust

- V těle funkce

```
1 /// GLOBAL_RNG_REGISTER is a valid memory mapped register
2 fn get_random() -> u64 {
3     unsafe {
4         *GLOBAL_RNG_REGISTER
5     }
6 }
```

Rust

# Kdy použít unsafe?

- Přístup k hardwarovým registrům (např. v embedded).
- Optimalizace výkonu kritických částí kódu.
- Rozhraní s C knihovnamí (FFI).
- Implementace nízkoúrovňových datových struktur (např. vlastní kolekce).
- Vždy by mělo být používání unsafe dobře zdokumentováno a izolováno do malých bloků.

# Pravidla pro unsafe

- Nečíst neinicializovanou paměť.
- Nepřekročit hranice pole nebo bufferu.
- Neprovádět data races - současný přístup z více vláken bez synchronizace.
- Všechny tyto podmínky musí být zachovány i v unsafe bloku – jinak může dojít k nedefinovanému chování.
- Borrow checker stále kontroluje, mutovatelný přístup z více míst lze jedině přes mutable ukazatel.

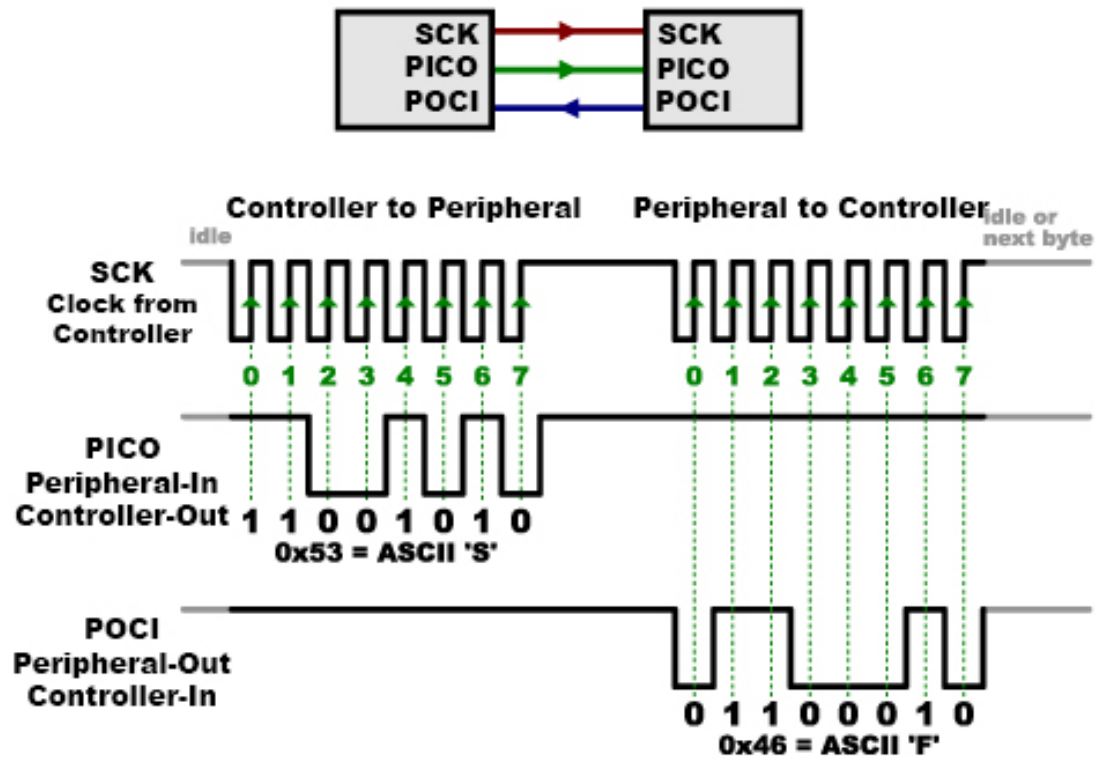
# Úvod do Embedded

- U embedded programování se bavíme od 8-bitových mikrokontrolérů s pár kilobajty RAM až po systémy s Gigabajty jako Raspberry Pi.
- Výhradně se bavíme o “bare-metal”, tj. váš program je jediné co běží na daném čipu.
- Nemáme žádnou kontrolu paměti kterou si explicitně nenastavíme v hardwaru. (pokud je to vůbec podporováno)
- Nemáme knihovny a ovládání periférii které nám dodává operační systém.

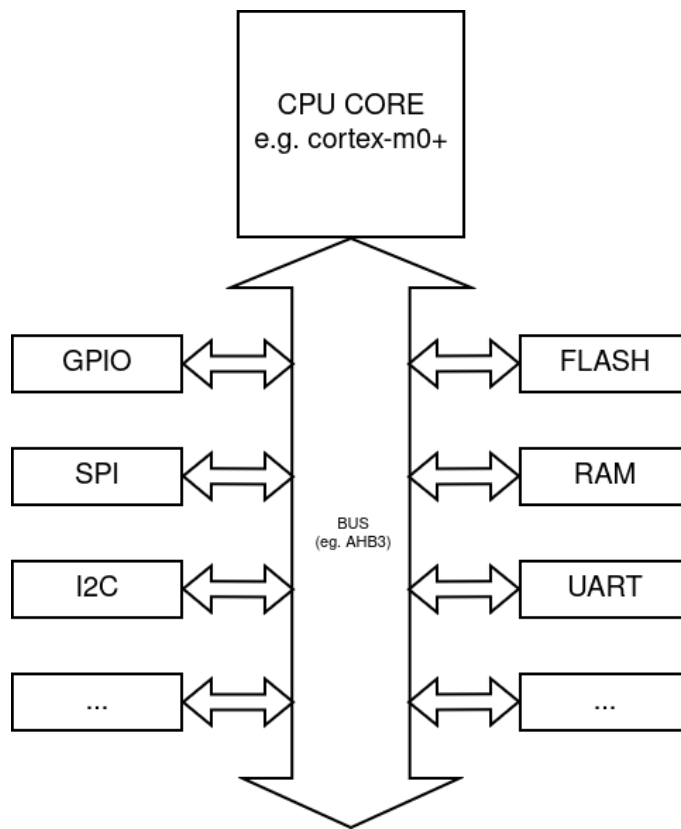
# Periférie

- Dá se říct že mikrokontrolér se skládá z několika částí.
- Processorové jádro - počítá a nastavuje.
- Periférie - dovolují interagovat s okolím světem ve formě přepínání pinů (GPIO, SPI, I2C, USB...) a pomáhá processorovému jádru v počítání (CRC32, přerušení, časovače...).
- Sběrnice - propojuje tyto části dohromady aby si processor myslel že je všechno v jeho adresním prostoru.

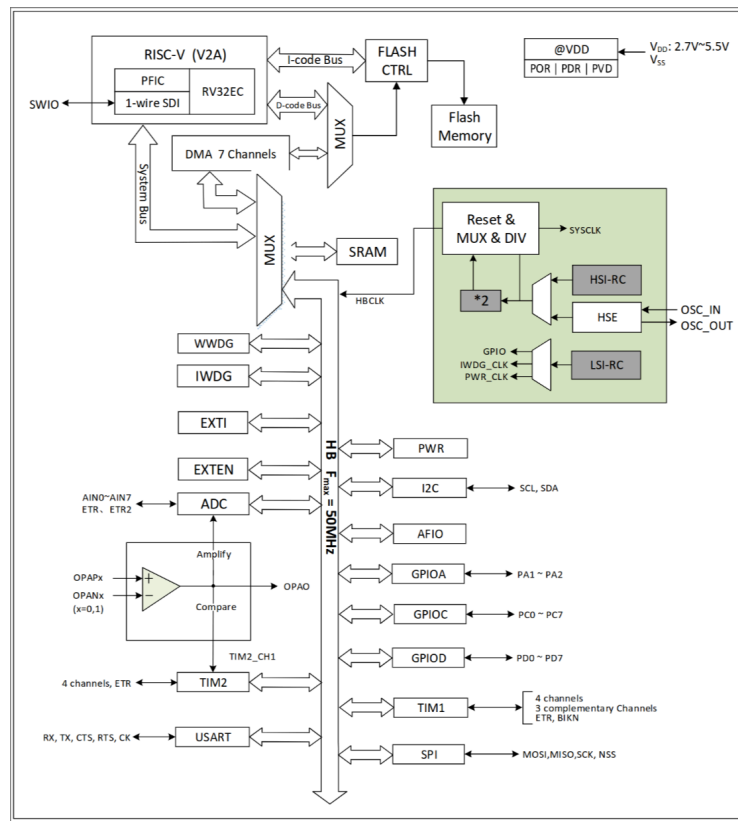
# SPI



# Základní model



# Příklad modelu (CH32v003)



# Příklad ovládacího registru

| IO_BANK0: GPIO0_CTRL, GPIO1_CTRL, ..., GPIO28_CTRL, GPIO29_CTRL |                                                                                         |      |       |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------|------|-------|
| Registers                                                       |                                                                                         |      |       |
| Offsets: 0x004, 0x00c, ..., 0x0e4, 0x0ec                        |                                                                                         |      |       |
| Description                                                     |                                                                                         |      |       |
| GPIO control including function select and overrides.           |                                                                                         |      |       |
| Bits                                                            | Description                                                                             | Type | Reset |
| 31:30                                                           | Reserved.                                                                               | -    | -     |
| 29:28                                                           | <b>IRQOVER</b>                                                                          | RW   | 0x0   |
|                                                                 | Enumerated values:                                                                      |      |       |
|                                                                 | 0x0 → NORMAL: don't invert the interrupt                                                |      |       |
|                                                                 | 0x1 → INVERT: invert the interrupt                                                      |      |       |
|                                                                 | 0x2 → LOW: drive interrupt low                                                          |      |       |
|                                                                 | 0x3 → HIGH: drive interrupt high                                                        |      |       |
| 27:18                                                           | Reserved.                                                                               | -    | -     |
| 17:16                                                           | <b>INOVER</b>                                                                           | RW   | 0x0   |
|                                                                 | Enumerated values:                                                                      |      |       |
|                                                                 | 0x0 → NORMAL: don't invert the peri input                                               |      |       |
|                                                                 | 0x1 → INVERT: invert the peri input                                                     |      |       |
|                                                                 | 0x2 → LOW: drive peri input low                                                         |      |       |
|                                                                 | 0x3 → HIGH: drive peri input high                                                       |      |       |
| 15:14                                                           | Reserved.                                                                               | -    | -     |
| 13:12                                                           | <b>OEOVER</b>                                                                           | RW   | 0x0   |
|                                                                 | Enumerated values:                                                                      |      |       |
|                                                                 | 0x0 → NORMAL: drive output enable from peripheral signal selected by funcsel            |      |       |
|                                                                 | 0x1 → INVERT: drive output enable from inverse of peripheral signal selected by funcsel |      |       |
|                                                                 | 0x2 → DISABLE: disable output                                                           |      |       |
|                                                                 | 0x3 → ENABLE: enable output                                                             |      |       |
| 11:10                                                           | Reserved.                                                                               | -    | -     |
| 9:8                                                             | <b>OUTOVER</b>                                                                          | RW   | 0x0   |
|                                                                 | Enumerated values:                                                                      |      |       |
|                                                                 | 0x0 → NORMAL: drive output from peripheral signal selected by funcsel                   |      |       |
|                                                                 | 0x1 → INVERT: drive output from inverse of peripheral signal selected by funcsel        |      |       |
|                                                                 | 0x2 → LOW: drive output low                                                             |      |       |
|                                                                 | 0x3 → HIGH: drive output high                                                           |      |       |

# Vývojový model

- Na “plných” operačních systémech je jasně dáno jak má binární program vypadat. (.exe, .elf)
- V embedded to nevíme, u každého čipu může program začínat jinde, může mít RAM jinde atd.
- Proto potřebujeme linker script.

# Linker script pro Rust program

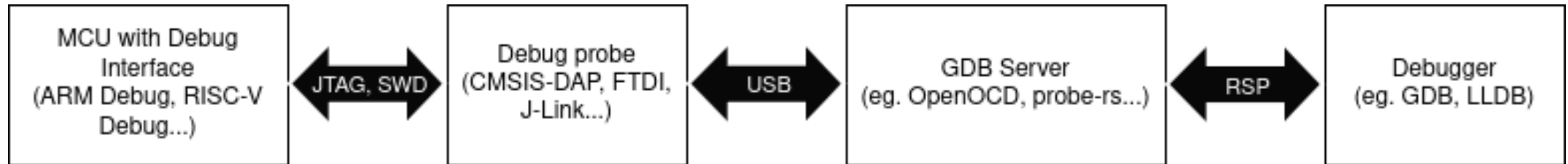
```
1  MEMORY
2  {.
3      RAM : ORIGIN = 0x2000, LENGTH = 8K
4      FLASH : ORIGIN = 0x0000, LENGTH = 8K
5  }
6
7  REGION_ALIAS("REGION_TEXT", FLASH);
8  REGION_ALIAS("REGION_RODATA", FLASH);
9  REGION_ALIAS("REGION_DATA", RAM);
10 REGION_ALIAS("REGION_BSS", RAM);
11 REGION_ALIAS("REGION_HEAP", RAM);
12 REGION_ALIAS("REGION_STACK", RAM);
```

ld

# Debugování embedded systémů

- Na PC běží program (OpenOCD/probe-rs) který předstírá že je debugovaný program.
- GDB/LLDB si tak myslí že debuguje normální program.
- OpenOCD či probe-rs však jen překládá debug příkazy z GDB/LLDB a posílá je dál.

# Debugování embedded systémů - příklad



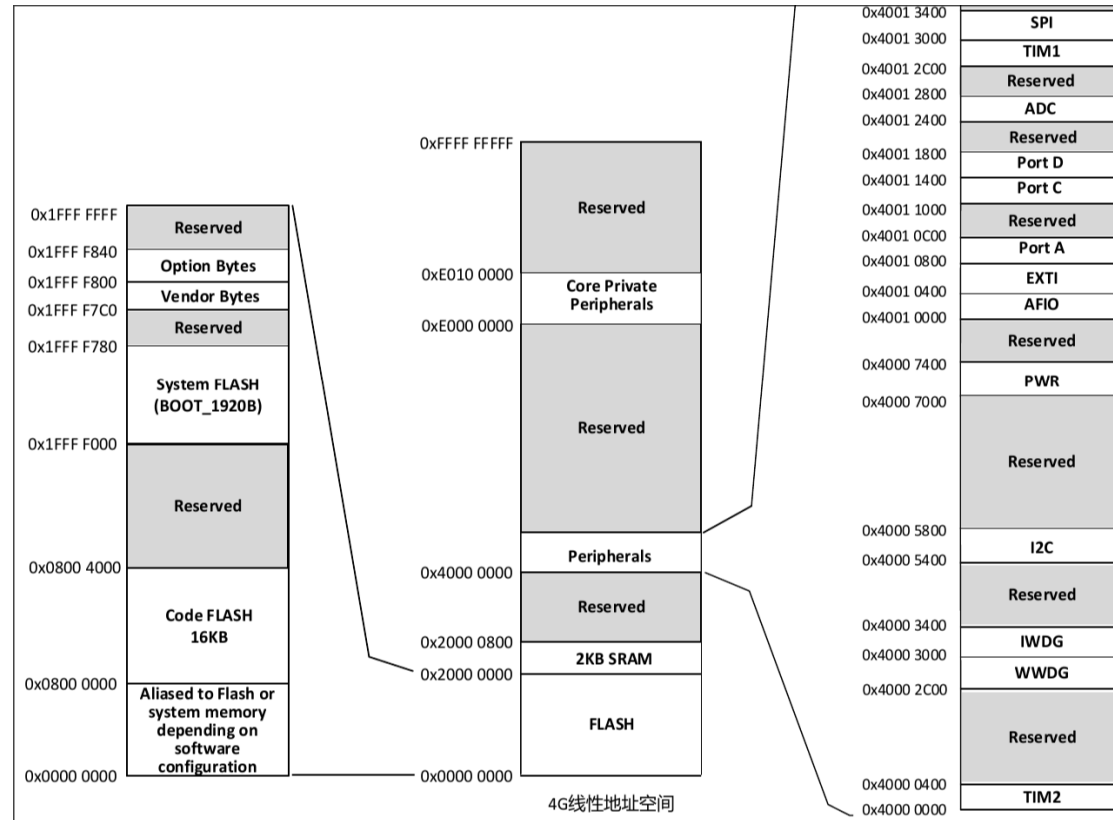
## **Embedded v C/C++**

- Instalovat si všechny dependence pro vývoj je pro běžného vývojáře těžké. Obzvláště v C++.
- Výrobci mají vlastní IDE kde si všechno řeší sami. Dovoluje to i jednodušší konfiguraci.
- Není na první pohled zřejmé jaké části standardní knihovny jsou či nejsou bezpečné pro embedded.
  - To stejné platí i pro většinu externích knihoven.
- Není sjednocený build systém, můžete být rádi když výrobce je schopen generovat projekty se CMake.

# Mapování ovládacích registru

- Ovládací registr = registr který ovládá nějaký aspekt processoru.
  - To je posílání dat, zapínání/vypínání/čtení pinů, spací režimy, konfigurace periférií...
- Registry jsou přímo mapovány do paměti. Jako kdyby to byla RAM.
- Když šáhneme na neplatnou paměť, v lepším případě dostaneme chybu dekódování adresy, v horším jsme si něco špatně nastavili.

# Mapování ovládacích registru - příklad



# Nastavení registrů přes nekontrolovaný přístup do adresního prostoru

```
1 fn main() {  
2     let mut ptr = 0x40014004usize as *mut u32;  
3     let enable_output = 0x3 << 12; // Enable output,  
    všechny ostatní bity na nule  
4     unsafe { ptr.write_volatile(enable_output); }  
5 }
```

Rust

- Velmi málokdy je třeba, většinou jsme schopni si vygenerovat vrstvu “PAC” z dat co dodává výrobce.

# **Embedded v Rustu**

## **no\_std**

- std knihovna v Rustu vyžaduje systémová volání od operačního systému pro řadu funkcionalit.
  - tj. operování se soubory, sockety, operování s vlákny, operování se vstupem/výstupem, atd.
- Knihovnu či aplikaci která nevyužívá standardní knihovny označíme pomocí `#![no_std]`
- U knihoven které podporují `no_std` si většinou musíte zapnout flag pro to.

## **core**

- Core knihovna v Rustu obsahuje funkcionalitu z `std` která nevyžaduje `dependence` či `alokace`.
- Stále máte `iterátory`, `Option`, `Result`, `Error` (od 1.81), `statická pole`, `Future`...
- Kolekce nejsou k dispozici, ty jsou v knihovně `alloc`.

# **alloc**

- Je třeba si nakonfigurovat alokátor aby šla používat alloc knihovna.
- V embedded se díky malému množství RAM a díky tomu že nemáme hardwarovou správu paměti snažíme nepotřebným alokacím vyhýbat.
- S dynamickou alokací je těžké předem předpovídat kolik paměti bude program potřebovat. Jen se statickou je to jednoduché.

# Kompilace a nahrání programu pro embedded

- Je třeba mít schopnost kompilovat pro danou architekturu kterou máme.
- Příklad, u RP2040:

```
1  $ rustup target add thumbv6m-none-eabi
```

sh

# .cargo/config.toml

```
1  [target.'cfg(all(target_arch = "arm", target_os = "none"))']
2  runner = "elf2uf2-rs -d"
3  rustflags = [
4      "-C", "linker=flip-link",
5      "-C", "link-arg=-nmagic",
6      "-C", "link-arg=-Tlink.x",
7  ]
8
9  [build]
10 target = "thumbv6m-none-eabi"
```

toml

**Ekosystém**

## **probe-rs**

- Software pro nahrávání a debugování embedded programů.
- Má v sobě podporu pro defmt (více později)
- Podporuje debug architektur ARM, Xtensa a RISC-V
- Alternativa k OpenOCD.
- Funguje i jako knihovna (narozdíl od OpenOCD).

# probe-rs - příklad

```
1 let mut session = Session::auto_attach(  
2     "RP2040",  
3     Permissions::default()  
4 )?;  
5  
6 // Select a core.  
7 let mut core = session.core(0)?;  
8  
9 // Read a block of 50 32 bit words.  
10 let mut buff = [0u32; 50];  
11 core.read_32(0x2000_0000, &mut buff)?;  
12  
13 // Writing is just as simple.  
14 let buff = [0u32; 50];  
15 core.write_32(0x2000_0000, &buff)?;
```

Rust

# PAC

- tzv. peripheral access crate, tj. knihovna pro přístup k registrům.
- Výrobci procesorů obvykle dodávají XML soubor který dané registry popisuje.
- Jsme schopni z něj generovat knihovnu pro přístup k registrům a modifikace jednotlivých bitů je jednodušší. (pomocí `svd2rust`)
- Staví se na něm HAL, tj. hardware abstraction layer.

## PAC - příklad

```
1 let mut pac = pac::Peripherals::take().unwrap();
```

Rust

```
2
```

```
3 pac.IO_BANK0.gpio(0).gpio_ctrl()
```

```
4     .modify(|w| w.oever().enable());
```

- V tomto případě se změní jen ty dva bity odpovídající OUTOVER.

# HAL

- Přístupovat k jednotlivým bitům stále bolí a je to nečitelné.
- K jednoduššímu přístupu máme knihovny které nám dávají větší abstrakci.
- Výrobci dodávají HAL v C. Ty Rustové jsou obvykle dělané komunitou.
  - espressif je jedna z mála firem která Rust podporuje
  - STM, Raspberry RP\*\*\*\* a Nordic Semiconductor jsou rozumně dobře podporované komunitou

# HAL - příklad

```
1  let mut out_pin = pins.gpio25.into_push_pull_output();
2
3  let mut in_pin = pins.gpio23.into_pull_down_input();
4
5  loop {
6      if in_pin.is_low().unwrap() {
7          out_pin.set_high().unwrap();
8      } else {
9          out_pin.set_low().unwrap();
10     }
11 }
```

Rust

## **embedded-hal a jeho varianty**

- Pro potřeby sdílení knihoven existují rozhraní které popisují často sdílenou funkcionalitu mezi čipy.
- tj. GPIO piny, SPI, I2C, CAN, UART...
- Každý čip toto implementuje jinak, proto by měl HAL tyto traits implementovat pro danou periférii.
- Knihovna pro obsluhování nějakého externího čipu tak může být jednoduše napsaná genericky.

# embedded-hal příklad

```
1 let spi_mosi =  
  pins.gpio7.into_function::<hal::gpio::FunctionSpi>();  
2 let spi_miso = pins.gpio4.into_function::<hal::gpio::FunctionSpi>();  
3 let spi_sclk = pins.gpio6.into_function::<hal::gpio::FunctionSpi>();  
4 let spi = hal::spi::Spi::<_, _, _, 8>::new(pac.SPI0, (spi_mosi,  
  spi_miso, spi_sclk));  
5 // Spi periférie implementuje trait SpiBus  
6 let mut spi = spi.init(&mut pac.RESETS, clocks.peripheral_clock.freq(),  
  16.MHz(), embedded_hal::spi::MODE_0);  
7 if spi.write(&[0]).is_ok() {  
8     // Success  
9 };
```

Rust

# Embassy

- Asynchronní runtime udělaný pro embedded.
- V některých případech nám dovoluje nepoužít RTOS.
- V případě že není co dělat, processor se uspí a čeká se na přerušení.
- Narozdíl od tokio jsou Future v embedded světě univerzální, nezávisí na runtime.

```

1  #[embassy_executor::main] Rust
2  async fn main(spawner: Spawner) {
3      // Initialize the embassy-rp HAL.
4      let p = embassy_rp::init(Default::default());
5
6      // Spawned tasks run in the background,
7      // concurrently.
8
9      spawner.spawn(blink(p.PIN_25.degrade())).unwrap(
10
11      let mut button = Input::new(p.P0_11, Pull::Up);
12      loop {
13          // Asynchronously wait for GPIO events
14          button.wait_for_low().await;
15          info!("Button pressed!");
16          button.wait_for_high().await;
17          info!("Button released!");
18      }
19  }

```

```

19 // Declare async tasks Rust
20 #[embassy_executor::task]
21 async fn blink(pin: AnyPin) {
22     let mut led = Output::new(pin, Level::Low,
23     OutputDrive::Standard);
24
25     loop {
26         // Timekeeping is globally available,
27         // no need to mess with hardware timers.
28         led.set_high();
29         Timer::after_millis(150).await;
30         led.set_low();
31         Timer::after_millis(150).await;
32     }
33 }

```

# RTIC

- Chceme runtime/RTOS se striktními garancemi kdy proběhne náš kód.
- Sami si řešíme přerušení.
- Compile time plánování tasků.
- Blíž k tradičním embedded projektům.

# RTIC - Příklad

```
1 #[shared]
2 struct Shared {}
3
4 #[local]
5 struct Local {
6     led: PA5<Output<PushPull>>,
7     state: bool,
8 }
9
10 #[init]
11 fn init(cx: init::Context) -> (Shared, Local) {
12     // Clock setup omitted ...
13
14     rtt_init_print!();
15     rprintln!("init");
16
17     // Setup LED
18     let mut gpioa = cx.device.GPIOA.split(&mut rcc.ahb);
19     let mut led = gpioa
20         .pa5
21         .into_push_pull_output(&mut gpioa.moder, &mut gpioa.otyper);
22     led.set_high().unwrap();
23
24     // Schedule the blinking task
25     blink::spawn().ok();
26
27     (Shared {}, Local { led, state: false })
28 }
```

Rust

```
1 #[task(local = [led, state])]
2 async fn blink(cx: blink::Context) {
3     loop {
4         rprintln!("blink");
5         if *cx.local.state {
6             cx.local.led.set_high().unwrap();
7             *cx.local.state = false;
8         } else {
9             cx.local.led.set_low().unwrap();
10            *cx.local.state = true;
11        }
12        Mono::delay(1000.millis()).await;
13    }
14 }
```

Rust

## defmt

- Při logování musíme tradičně dávat celé řetězce do bináry. Na embedded chceme šetřit.
- Místo celého řetězce jen tedy pošleme ID řetězce a případně hodnoty pro formátování. Formátovací řetězce necháme v `.elf` souboru a debugger si je přečte a formátuje za nás.
- Je třeba nějak transportovat ID řetězců a hodnoty pro formátování.

```
1 // [3, 44, 1]
```

Rust

```
2 defmt::error!("The answer is {=i16}!", 300);
```

# defmt - pokračování

Otázka je jak transportovat tyto data z embedded čipu na PC, je několik možností.

`defmt-rtt` - Vyhradí se část RAM kterou si za běhu bude debugger číst. Hardware to musí podporovat.

`defmt-itm` - Hardware loguje přes speciální periférii pro to určenou, jen u některých ARM čipů.

`defmt-semihosting` - Hardware se zastaví a čeká než si to debugger nepřečte, nedoporučeno.

`defmt-bbq` - Loguje se do interního bufferu, čeká se že si transport vyřešíte sami

# Heapless

- Knihovna pro statické struktury, vhodné pro embedded.
- Častokrát potřebujeme pro čistotu kódu rozhraní vektoru nebo hašovací tabulky, heapless nám ho dá, za cenu že musíme předem říct kolik místa maximálně potřebujeme.
- Data tak jdou na zásobník, ne na haldu.

# Heapless - příklad

```
1 let mut vec = Vec::<i32, 8>::new();
```

Rust

```
2 vec.push(1);
```

```
3 vec.push(2);
```

```
4
```

```
5 assert_eq!(vec.len(), 2);
```

```
6 assert_eq!(vec[0], 1);
```

```
7
```

```
8 assert_eq!(vec.pop(), Some(2));
```

```
9 assert_eq!(vec.len(), 1);
```

# Postcard

- Embedded serializer a deserializer pro serde.
- Důraz na šetření zdrojů, i při přenosu dat.
- Dovoluje vám sdílet struktury mezi host počítačem a embedded čipem.
- Součástí je i parsování dat a překlad z bitového streamu na struktury.  
(`postcard::accumulator::CobsAccumulator`)

# Postcard - příklad

```
1  #[derive(Serialize, Deserialize, Debug, Eq, PartialEq)]
2  struct RefStruct<'a> {
3      bytes: &'a [u8],
4      str_s: &'a str,
5  }
6  let message = "hElLo";
7  let bytes = [0x01, 0x10, 0x02, 0x20];
8  let output: Vec<u8, 11> = to_vec(&RefStruct {
9      bytes: &bytes,
10     str_s: message,
11 }).unwrap();
```

Rust

**Dotazy?**

**Děkuji za pozornost**